

Project Setup and Source Control

- **Tool:** Git / GitHub
- **Task:** Initialize the project repository and organize files.
- **How-To:**
 - **Create a GitHub repository:** On GitHub, create a new repository (e.g. *hotel-analytics-portfolio*). Clone it locally or connect your IDE.
 - **Organize the folder structure:** In your local repo, create directories like `data/raw/`, `data/clean/`, `scripts/`, and `docs/`. Copy the raw CSV files into `data/raw/`.
 - **Initialize Git:** In your project root, run `git init`, then `git add .` and `git commit -m "Initial project structure with raw data"`.
 - **Push to GitHub:** Add the remote (`git remote add origin <your-repo-URL>`) and `git push -u origin main`. Use a `.gitignore` to exclude large intermediate files (e.g. logs or temporary caches).
 - **Versioning:** As you proceed through data work and code development (in Python, SQL, Power BI templates), commit changes frequently with descriptive messages (e.g. "Clean fact_bookings and load into Postgres"). This ensures an auditable history of your analysis pipeline.

Phase 1: Data Inventory and Profiling (Tool: Python)

- **Task:** Inspect each raw CSV to understand schemas, volumes, and data quality.
- **How-To:**
 - **Load data with Pandas:** In a Jupyter notebook or Python script, use `pd.read_csv()` on each CSV (`fact_bookings.csv`, `fact_aggregated_bookings.csv`, `dim_hotels.csv`, `dim_rooms.csv`, `dim_date.csv`). For example:

```
import pandas as pd
df_bookings = pd.read_csv('data/raw/fact_bookings.csv')
df_hotels    = pd.read_csv('data/raw/dim_hotels.csv')
# ...and so on for each file
```

- **Examine schemas:** Use `df.info()` and `df.columns` to list each dataset's columns and data types. For example, in `fact_bookings` we see columns like `booking_date`, `check_in_date`, `no_guests`, `booking_status`, `revenue_generated`, etc. Note any abnormal data types (e.g. dates loaded as strings).
- **Preview records:** Use `df.head()` to see sample rows. Check formatting of dates (`27-04-2022`), numeric formats, and categorical values. For instance, in `fact_bookings` you might notice `no_guests` has negative values (likely a data error) and `ratings_given` has many missing entries (since only some guests rate their stay).
- **Profile data quality:** Compute basic stats: `df.describe()` for numeric columns, `df['column'].value_counts()` for categorical fields. For example, count the number of "Checked Out" vs "Cancelled" bookings (`df_bookings['booking_status'].value_counts()`).

Check for nulls with `df.isnull().sum()`. Note that `ratings_given` is mostly null (it can be handled or dropped if not needed), and `fact_aggregated_bookings.capacity` may have a few missing values.

- **Document issues:** Record any data issues (negative guests, missing values, inconsistent text like mixed case in `booking_platform`, etc.). This step ensures you understand what cleaning steps are needed next.

Phase 2: Data Cleaning and Transformation (Tool: Python)

- **Task:** Clean anomalies, standardize formats, and prepare “analysis-ready” tables.
- **How-To:**
- **Parse dates:** Convert date strings to Pandas datetime. For example:

```
df_bookings['booking_date'] = pd.to_datetime(df_bookings['booking_date'],
dayfirst=True)
df_bookings['check_in_date'] =
pd.to_datetime(df_bookings['check_in_date'], dayfirst=True)
df_bookings['checkout_date'] =
pd.to_datetime(df_bookings['checkout_date'], dayfirst=True)
```

Do the same for the `check_in_date` in `fact_aggregated_bookings` and the `date` column in `dim_date`. This enables date-based calculations and ensures consistency.

- **Fix numeric anomalies:** Address the negative `no_guests` values. Decide on a rule: if the negative indicates a data entry error, you might take absolute values or set them to a minimum of 1. Alternatively, drop bookings with invalid guest counts if they are few. For example:

```
df_bookings = df_bookings[df_bookings['no_guests'] > 0]
# drop negative or zero guest entries
```

- **Handle missing values:**
 - For `ratings_given`, you may leave as is or fill nulls with a sentinel (e.g. 0 or "No Rating") if needed for analysis. Since this KPI isn't central, it can be ignored or dropped.
 - In `fact_aggregated_bookings`, fill missing `capacity` either by using a default (e.g. infer from similar dates/hotels) or drop those rows if they are very few. You might use `df_agg['capacity'].fillna(method='ffill')` or manual imputation.
- **Standardize categories:** Clean text fields for consistency. For example, strip whitespace and unify case:

```
df_bookings['booking_platform'] =
df_bookings['booking_platform'].str.strip().str.title()
```

Check `dim_hotels['category']` or `df_hotels['city']` for consistent naming. If any typos or inconsistent spellings appear, correct them.

- **Compute derived fields:** Add useful columns:
 - **Length of stay (nights):** Compute as `(checkout_date - check_in_date).days` for each booking.
 - **Year/Month columns:** From `booking_date` or `check_in_date`, extract year and month for easier grouping (e.g. `df_bookings['booking_month'] = df_bookings['booking_date'].dt.to_period('M')`).
 - **Booking status flags:** Create indicator columns if needed (e.g. `is_cancelled = df_bookings['booking_status'] == 'Cancelled'`).
- **Filter for analysis:** Depending on your KPIs: keep all records for analysis, but know which status means what. For instance, when calculating occupancy or revenue, use only `Checked Out` (completed) bookings. For the cancellation rate, count `Cancelled` (and possibly `No Show`) against all bookings.
- **Save cleaned data:** Write the cleaned tables to new CSVs (e.g. `data/clean/fact_bookings_clean.csv`, etc.) using `df.to_csv()`. This preserves the raw originals and provides sanitized inputs for the database load.

Phase 3: Data Modeling in PostgreSQL (Tool: PostgreSQL)

- **Task:** Define a relational schema (star schema) and load the clean data into a PostgreSQL database.
- **How-To:**
 - **Design the schema:** Create a star schema with one or more fact tables and shared dimensions. In our case: one fact table for detailed bookings (`fact_bookings`) and one aggregated fact table (`fact_aggregated_bookings`); plus dimension tables: `dim_hotels`, `dim_rooms`, `dim_date`. *Figure: A star schema has a central fact table connecting to various dimension tables* ¹.

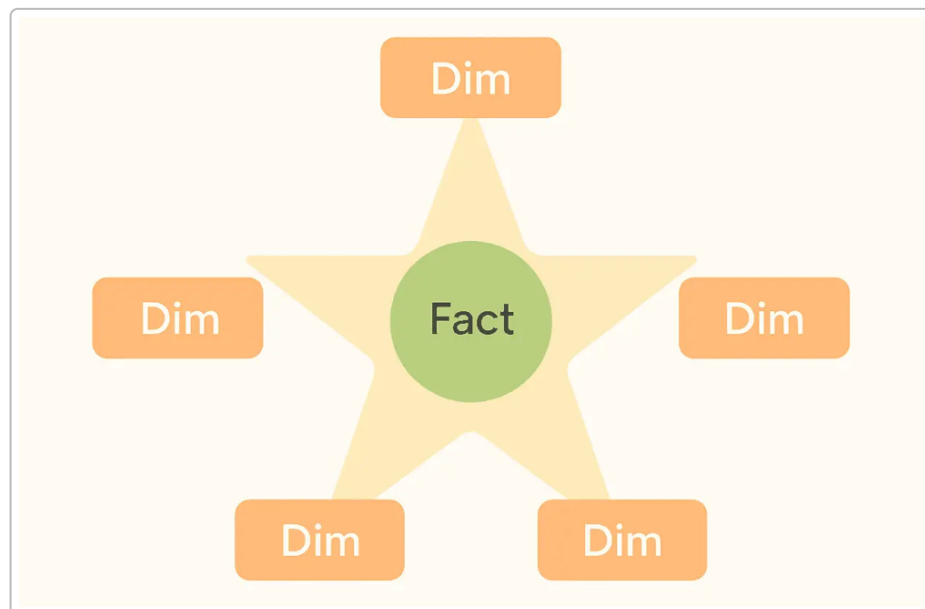


Figure: In a star schema, a central Fact table links to multiple Dimension tables ¹.
 In our design, the `fact_bookings` table will link (via foreign keys) to `dim_hotels`, `dim_rooms`, and `dim_date` tables. This denormalized design (optimized for analytics) matches the star-schema approach ¹.

- **Define tables in SQL:** Using a SQL client (e.g. `psql`, pgAdmin, or a Python DB connection), execute CREATE TABLE statements. For example:

```
CREATE TABLE dim_hotels (  
    property_id    INTEGER PRIMARY KEY,  
    property_name  TEXT,  
    category       TEXT,  
    city           TEXT  
);  
CREATE TABLE dim_rooms (  
    room_id    TEXT PRIMARY KEY,  
    room_class TEXT  
);  
CREATE TABLE dim_date (  
    date        DATE PRIMARY KEY,  
    month_yyyy  TEXT,  
    week_no     TEXT,  
    day_type    TEXT  
);  
CREATE TABLE fact_bookings (  
    booking_id      TEXT PRIMARY KEY,  
    property_id     INTEGER REFERENCES dim_hotels(property_id),  
    booking_date     DATE REFERENCES dim_date(date),  
    check_in_date    DATE REFERENCES dim_date(date),  
    checkout_date    DATE,  
    no_guests        INTEGER,  
    room_category    TEXT REFERENCES dim_rooms(room_id),  
    booking_platform TEXT,  
    ratings_given    INTEGER,  
    booking_status   TEXT,  
    revenue_generated NUMERIC,  
    revenue_realized NUMERIC  
);  
CREATE TABLE fact_aggregated_bookings (  
    property_id     INTEGER REFERENCES dim_hotels(property_id),  
    check_in_date    DATE REFERENCES dim_date(date),  
    room_category    TEXT REFERENCES dim_rooms(room_id),  
    successful_bookings INTEGER,  
    capacity         INTEGER  
);
```

Ensure foreign keys match data types (e.g. `property_id` INT, `room_category` TEXT matching `dim_rooms.room_id`). Adjust types if needed (e.g. if IDs should be VARCHAR).

- **Create and connect to the database:** If not already done, create a new Postgres database (e.g. `hotel_analytics`). Connect via a client or Python's `psycopg2` / `sqlalchemy`.

- **Load dimension tables:** Use the SQL `COPY` command or GUI import to load CSVs into dims:

```
COPY dim_hotels(property_id, property_name, category, city)
FROM '/path/to/data/clean/dim_hotels.csv' DELIMITER ',' CSV HEADER;
```

Repeat for `dim_rooms` and `dim_date`. For `dim_date`, make sure the `date` column is in `YYYY-MM-DD` or properly cast.

- **Load fact tables:** Similarly load `fact_bookings_clean.csv` and `fact_aggregated_bookings_clean.csv`. For example:

```
COPY fact_bookings(...) FROM '/path/to/data/clean/fact_bookings_clean.csv'
CSV HEADER;
COPY fact_aggregated_bookings(...) FROM '/path/to/data/clean/
fact_aggregated_bookings_clean.csv' CSV HEADER;
```

Verify that all rows loaded and constraints hold (no orphan foreign keys).

- **Validate and index:** Run simple queries to check counts (e.g. `SELECT COUNT(*) FROM fact_bookings;`). Optionally create indexes on frequently joined columns (e.g. an index on `fact_bookings.booking_date` or `property_id`) to speed up Power BI queries.
- **Refine schema:** If needed, you can add surrogate keys (e.g. a unique ID for `dim_date` if not using actual date as PK) or further normalize. But the above star model (one fact connecting to denormalized dims) is standard for decision support ¹.

Phase 4: Data Analysis and Visualization (Tool: Power BI)

- **Task:** Build a Power BI data model, create measures (KPIs), and design interactive dashboards answering key business questions.
- **How-To:**
- **Connect to data:** In Power BI Desktop, use **Get Data** → **PostgreSQL** (or CSV). Enter your database credentials and select the `dim_` and `fact_` tables. Import or DirectQuery as appropriate (import is simpler for static analysis).
- **Create relationships:** In the Model view, connect the fact tables to dimensions:
 - Link `fact_bookings.booking_date` to `dim_date.date` (1:many, one `dim_date` per date).
 - Link `fact_bookings.check_in_date` to `dim_date.date` similarly.
 - Link `fact_bookings.property_id` to `dim_hotels.property_id`.
 - Link `fact_bookings.room_category` to `dim_rooms.room_id`.
 - Do the same for `fact_aggregated_bookings`: join its `check_in_date` to `dim_date.date`, `property_id` to `dim_hotels`, and `room_category` to `dim_rooms`. Ensure relationship cardinalities are correct (one dim to many fact). This star schema (illustrated above) enables efficient slicing by date, hotel, and room class.
- **Define calculated measures (DAX):** In the Report view, create measures for your KPIs:
 - **Occupancy Rate:** Use the aggregated bookings data. For example:

```
Occupancy% = DIVIDE(
    SUM(fact_aggregated_bookings[successful_bookings]),
```

```

SUM(fact_aggregated_bookings[capacity]),
0
)

```

Format as percentage. This follows the formula: *Occupied Rooms ÷ Available Rooms × 100* ² .

- **Average Daily Rate (ADR):** Calculate total room revenue over rooms sold. If each booking in `fact_bookings` is one room-night, you can do:

```

ADR = DIVIDE(
    SUM(fact_bookings[revenue_realized]),
    COUNTROWS( fact_bookings ),
    0
)

```

(Alternatively use `SUM(fact_bookings[revenue_realized]) / SUM(fact_aggregated_bookings[successful_bookings])` . The Chekin reference gives $ADR = Total\ Room\ Revenue \div Rooms\ Sold$ ³ .)

- **Revenue Per Available Room (RevPAR):** Two equivalent methods:

```

RevPAR = DIVIDE(
    SUM(fact_bookings[revenue_realized]),
    SUM(fact_aggregated_bookings[capacity]),
    0
)

```

Or simply: `RevPAR = [ADR] * [Occupancy%]` . (As Chekin notes: $RevPAR = Room\ Revenue \div Rooms\ Available$ or $ADR \times Occupancy$ ⁴ .)

- **Cancellation Rate:** If you want a measure for cancellations:

```

Cancellation% =
    DIVIDE(

    CALCULATE(COUNTROWS(fact_bookings), fact_bookings[booking_status] =
    "Cancelled"),
    COUNTROWS(fact_bookings),
    0
    )

```

This follows $Cancelled\ Reservations \div Total\ Reservations \times 100$ ⁵ . Format as percentage.

- **Average Length of Stay (ALOS):** If needed:

```

ALOS =
    DIVIDE(
        SUMX(fact_bookings, DATEDIFF(fact_bookings[check_in_date],

```

```
fact_bookings[checkout_date], DAY)),
    COUNTROWS(fact_bookings),
    0
)
```

(This equals *Total Room Nights ÷ Number of Bookings* ⁶.)

- **Create visuals:** Build a dashboard answering key questions:
 - **Time trends:** Use line charts showing *Occupancy %*, *ADR*, and *RevPAR* over time (monthly or weekly). Drag the date field from **dim_date** (e.g. month-year) on the X-axis and the measures on Y. This shows seasonal patterns.
 - **Comparison by Hotel/City:** Use bar or column charts to compare KPIs across dimensions. For example, *RevPAR* by **dim_hotels[city]** or by **dim_hotels[category]** (hotel star category). Add slicers or filters to isolate specific time periods.
 - **Room category analysis:** Show room revenue or occupancy by room_class (from **dim_rooms**) to see which classes drive revenue.
 - **Cancellation insight:** Create a card or gauge to display **Cancellation%**. You could also show cancelled vs booked counts in a bar chart by booking platform (using **booking_platform** from **fact_bookings**) to see which channel has the highest cancellations.
 - **KPI cards:** Use Power BI's KPI or Card visuals to highlight single values (e.g. Current Month *Occupancy %*, *ADR*, *RevPAR*). Add indicators or trend arrows by comparing to previous period.
 - **Interactive filters:** Add Slicers for date range, city, hotel category, or booking platform so stakeholders can drill into segments.
- **Refine and format:** Apply a clean color theme and meaningful labels. Ensure your axis titles and data labels are clear (e.g. "Occupancy %"). If needed, add tooltips or annotations explaining KPIs.
- **Publish and iterate:** Upload the report to Power BI Service if desired. Use version control by exporting PBIX files or using the Power BI Git integration. Iterate visuals based on feedback (e.g. add a map for geo-analysis if city lat/long added).

Phase 5: Presentation Structure (Tool: Gemma AI)

- **Task:** Generate a slide deck outline summarizing the project and insights.
- **How-To:**
 - **Access Gemma (Gamma) AI:** Log into Gemma (also known as Gamma.app) and create a new presentation. Choose the option to generate a deck from text.
 - **Provide a prompt:** Instruct Gemma to outline the presentation. For example: "Create an 8-slide presentation outline for a hotel revenue analysis project. Include objectives, data sources, methodology (data cleaning, modeling), key KPIs (occupancy rate, ADR, RevPAR, cancellation rate), findings, and recommendations."
 - **Review the generated structure:** Gemma will propose slide titles and bullet points. You might get slides like: Introduction (objectives), Data Description, Methodology (tools used: Python, Postgres, Power BI), KPIs and Questions, Analysis Findings, Conclusion.
 - **Customize slides:** Adjust wording or add slides as needed. For example, add a slide specifically for "Data Cleaning" or one for "Power BI Dashboard Highlights". Provide additional instructions if the outline misses a critical part.
 - **Add visuals:** In Gemma, you can upload or link charts from Power BI to each slide. For instance, insert the *Occupancy Rate* trend line on the "Key Metrics" slide, and a bar chart of *RevPAR* by city on the "Insights" slide. Use Gemma's AI suggestions for relevant icons or images.

- **Polish design:** Use Gemma's formatting to apply a consistent theme, align content, and ensure readability. Each slide's bullets should be concise (3–5 points).
- **Export final deck:** When satisfied, export the deck as a PowerPoint or PDF. Commit this file (or key outline notes) to GitHub under `docs/` for your portfolio.

Key Business Questions & KPIs

To make this a decision-support dashboard, focus on *actionable hotel management metrics*. Below is a prioritized list of industry-standard questions and their associated KPIs (with formulas):

1. What is the *Occupancy Rate* by hotel (or city) over time?

2. *KPI:* $\text{Occupancy (\%)} = (\text{Rooms Occupied} \div \text{Rooms Available}) \times 100$ ² .

3. *Purpose:* Indicates how effectively inventory is used. (E.g., an 80% occupancy suggests strong demand in that period ² .) Track this daily/weekly to spot seasonal peaks.

4. What is the *Average Daily Rate (ADR)* of rooms?

5. *KPI:* $\text{ADR} = \text{Total Room Revenue} \div \text{Number of Rooms Sold}$ ³ .

6. *Purpose:* Reflects average price achieved per occupied room. Higher ADR improves revenue if occupancy stays constant. Compare ADR across room types or seasons to assess pricing strategies ³ .

7. What is *RevPAR (Revenue per Available Room)* across hotels?

8. *KPI:* $\text{RevPAR} = \text{ADR} \times \text{Occupancy\%}$ (or $\text{Room Revenue} \div \text{Rooms Available}$) ⁴ .

9. *Purpose:* Combines price and occupancy into one metric, key for revenue management ⁴ . High RevPAR means the hotel maximizes revenue from its inventory. Use it to benchmark performance between properties.

10. What is the *Cancellation Rate* (or *No-Show Rate*)?

11. *KPI:* $\text{Cancellation Rate (\%)} = (\text{Cancelled Reservations} \div \text{Total Reservations}) \times 100$ ⁵ .

12. *Purpose:* High cancellation rates indicate lost revenue and inefficiencies. By platform or channel, it informs policy changes or overbooking strategies ⁵ .

13. How does *Average Length of Stay (ALOS)* trend?

14. *KPI:* $\text{ALOS} = \text{Total Room Nights Sold} \div \text{Number of Bookings}$ ⁶ .

15. *Purpose:* Longer stays typically improve occupancy stability. This metric helps forecast staffing and cross-sell opportunities for guests.

16. **Which *hotel categories* or *room types* are most profitable?**

17. *KPI*: (Derived) Compare RevPAR and ADR by `dim_hotels[category]` or `dim_rooms[room_class]`.

18. *Purpose*: Identifies segments (e.g. Luxury vs Budget, or Suite vs Standard rooms) that drive revenue. Guides inventory and marketing focus.

19. **Trend of *Revenue* and *Bookings* over time (seasonality).**

20. *KPI*: Total revenue and bookings by month/quarter.

21. *Purpose*: Reveals peak seasons or off-peak lows. Supports planning (e.g. promotions during low occupancy periods).

22. **Customer Feedback (if available):**

23. *KPI*: Average rating by hotel.

24. *Purpose*: Ensures high occupancy/ADR aren't harming guest satisfaction. (This project has `ratings_given`; showing it alongside RevPAR could be insightful.)

Each of the above KPIs should be visualized (e.g. line charts for trends, bar charts for comparisons, KPI cards for current values) in your Power BI dashboard so stakeholders can quickly assess performance. These metrics are industry-standard for hotel revenue management and address the core decision questions: *How full are we, how much do we charge, how much do we earn per room, and are cancellations impacting our business* ² ⁵.

Sources: Industry definitions of these metrics are well-documented. For example, the Chekin blog defines Occupancy, ADR, RevPAR, and Cancellation formulas ² ⁵, while data warehousing best practices describe using a star schema for analytics ¹.

¹ Star Schema Guide: Data Warehouse Modeling Explained

<https://motherduck.com/learn/star-schema-data-warehouse-guide/>

² ³ ⁴ Hotel occupancy rate: how to calculate it and improve profitability - Chekin

<https://chekin.com/en/blog/hotel-occupancy-rate/>

⁵ ⁶ Hotel KPIs: Top 10 Metrics Every Hotelier Should Track - Chekin

<https://chekin.com/en/blog/hotel-kpis-top-10-metrics-every-hotelier-should-track/>